ComEng 2 - Ergänzung

HS 2026 – Prof. Lars Kamm

Autoren:
Mirko Ratti

Version:
1.0.20261015
https://github.com/mirkoratti/HS2026

Inhaltsverzeichnis

14 Arm Cortex-M4 & Timer **21**
14.1 Watchdog Window-Watchdog-Timer (WWDT) . . . 21
14.2 Spezialtimer STM32G4 . . . 21
14.3 M4 vs M0 . . . 21
14.4 Wichtigkeit Standardisierung von ARM-Cortex . . . 21

15 System & Speicher **21**
15.1 Bestandteile . . . 21
15.2 Vorteile des Cortex-M4 NVIC . . . 21
15.3 Infos über den D-Code Systembus . . . 21

16 Architektur & Befehle **21**
16.1 ISA Thumb-2 Befehlssatz . . . 21
16.2 Adressing Modes . . . 21

17 Floating Point Unit (FPU) **21**
17.1 IEEE 754: Grundlagen . . . 21
17.2 IEEE 754: Addition / Subtraktion . . . 21
17.3 IEEE 754: Moltiplicazione / Divisione . . . 22

18 Schnittstellen & Peripherie **22**
18.1 uC-Peripherie . . . 22
18.2 Schieberegister . . . 22
18.3 Taktgeber in einem uP-System . . . 22
18.4 Direct Memory Access (DMA) . . . 22
18.5 UART . . . 22
18.6 SPI . . . 22
18.7 I2C . . . 22

19 Anhang **22**

14 Arm Cortex-M4 & Timer

14.1 Watchdog Window-Watchdog-Timer (WWDT)

- Zur Überwachung der Code-Ausführung. Erzeugt im Fehlerfall einen Reset.
- Dieser Timer muss jeweils vor Ablauf der Timeout-Zeit zurückgesetzt werden.
- Ansonsten wird eine definierte Aktion ausgeführt (z.B. Reset).

14.2 Spezialtimer STM32G4

HRTIM (high resolution)	LPTIM (low power)
RTC	WWDG (eigenen Code überwachen)

14.3 M4 vs M0

- **M4:** schnellere Taktung als M0 ⇒High Performance / neuere Technologie
- **M0:** Low Power / Low Cost (alte Technologie)

14.3.1 Cortex-M4 Prozessor

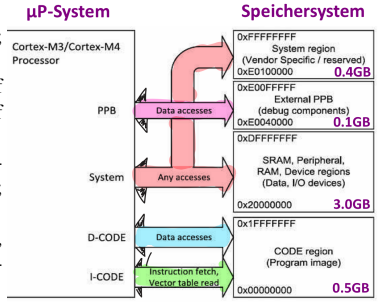
- 3-stufige Pipeline für Thumb-2 Instruktionssatz
- NVIC, SYSTICK & leistungsfähige debug Units
- Memory Protection Unit (MPU) und Floating Point Unit (FPU)
- Erweiterte Busstruktur

  I-Code Bus Instruction- & Vector-Fetch; 32-Bit-Zugriff (Thumb-1).

  D-Code Bus Daten- & Debugzugriff auf ROM (Konstanten); unaligned Zugriff möglich (wird in aligned gewandelt).

  System Bus Zugriff auf SRAM & Peripherie (Variablen); für Code & Daten; unaligned Zugriff möglich.

  PP Bus Privater Peripheriebus (Debug, MPU, FPU); nur word-aligned Transfer.



14.3.2 Vorteile Cortex-M4 gegenüber Cortex-M0

- Floating Point Unit, erweiterte Debug-Unit
- Harvard-Architektur
- Bit-Banding, SIMD-Befehle, erweiterter Befehlssatz, DSP-Funktionen.
- Schnellere Ausführung von Code, unterstützt Thumb-2.
- ALU und Barrel Shifter und Multiply & Accumulate (MAC)

14.4 Wichtigkeit Standardisierung von ARM-Cortex

- Verwendung des gleichen Cores durch verschiedene Chip-Hersteller.
- Es gibt eine Abwärtskompatibilität innerhalb der Familien.

15 System & Speicher

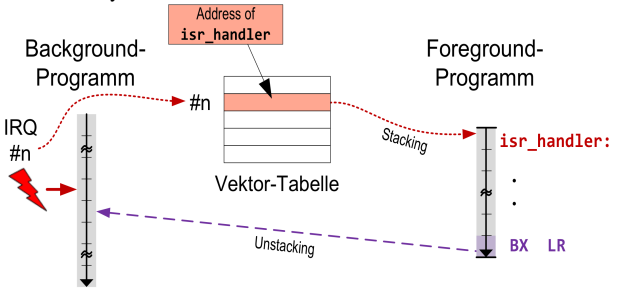
15.1 Bestandteile μC

Hauptbestandteile
- CPU (μP), Speicher (ROM, RAM), Systemkomponenten (Clock).
- Peripherie-Elemente (GPIO, ADC, DAC, Timer, SPI, I2C, . . .)

Grundelemente
- Stromquelle, POR, Taktgeber & Boot-Code (init of internal system)

15.2 Vorteile des Cortex-M4 NVIC

- Kein Software-Overhead für Stacking und Sprünge.
- Parallele Verarbeitung von Code und Daten (Harvard).
- Sehr geringe Jitter-Zeiten bei der Ausführung.
- Nur 12 Cycles für Ein- und Austritt der ESR bzw. ISR.



15.3 Infos über den D-Code Systembus

- Datenzugriff für ROM (Flash schreiben).
- Debug-Infos aus dem ROM (Code) auslesen.

[Placeholder: Speichertabelle (LA 1-28 Werte)]

16 Architektur & Befehle

16.1 ISA Thumb-2 Befehlssatz

- Verarbeitet 32 und 16 Bit Befehle, Code-Density, Performance, mehr Befehle möglich.
- Flexibler 2. Operand <op2>
- Erweiterte Addressing-Modes
- Erweiterte Grundoperationen (z.B. OR NOT, für jeden 16Bit-Befehl neu auch einen 32Bit-Befehl)
- Bit-Field Processing
- Compare & Branch Zero, Conditional Execution (IT-Instruction)
- Saturation (32Bit)
- Multiply & Accumulate (32Bit MAC)

16.1.1 Single Instruction Multiple Data (SIMD)

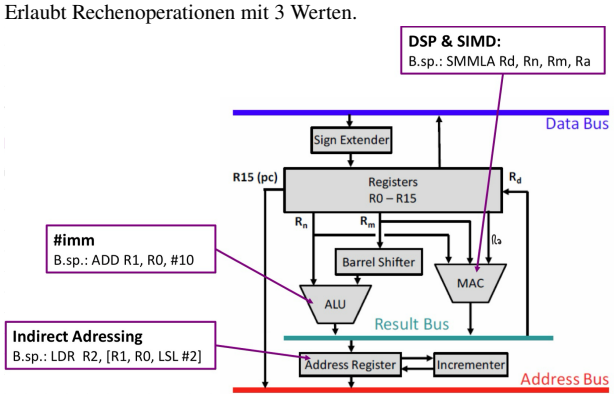
- ganze Arrays von Daten werden übergeben.
- Saturation (8/16Bit)
- Multiply & Add/Sub oder Accumulate (8/16Bit)
- Add & Sub (8/16Bit)
- Add/Sub & Saturation

16.1.2 Load/Store-Architektur (RISC)

Sign Extender, Barrel-Shifter (Erlaubt Shift-Operationen auf R).

16.1.3 Multiply & Accumulate (MAC)

Erlaubt Rechenoperationen mit 3 Werten.



16.2 Adressing Modes

Mode	**Beispiel**	**Effekt** (A = Zugriffsadresse)	
Direkt	`R1, #100`	Wert direkt im Befehl/Reg.	
Liste	`{R1-R3, R5, LR}`	mehrere Reg. (PUSH/POP, STM)	
Indirekt	`[R0]`	A = R0	
Indirekt Offset	`[R0, #4]`	A = R0 + 4	(kein Write-back)
Indirekt Index	`[R0, R1]`	A = R0 + R1	
Pre-Index*	`[R0, #4]!`	zuerst R0 += 4, dann A = R0	(C: `x = *++p`)
Post-Index*	`[R0], #4`	zuerst A = R0, dann R0 += 4	(C: `x = *p++`)
Shifted Index*	`[R0, R1, LSL #2]`	A = R0 + 4 · R1	
PC-relativ	`label, [PC, #4]`	A = PC + 4 (Konstanten)	
SP-relativ	`[SP, #4]`	A = SP + 4 (lokale Var.)	

* neu ab M3/M4. ! = **Write-back**: Basisregister wird überschrieben.

Kombination (Barrel Shifter): `LDR R1, [R0, R2, LSL #8]!` ⇒ R1 = R0 + 256 · R2, mit Write-back.

17 Floating Point Unit (FPU)

- Program Control & Status Register (FPCSR)
- Grundoperationen: Mathematische Befehle, Datentransferbefehle, Konvertierungsbefehle, Compare-Befehl

Register FPU

32 Single Precision Register (S0-S31) 16 Double Precision Register (D0-D15)

17.1 IEEE 754: Grundlagen

Nutzen Normalisierung der Mantisse
- Höhere Genauigkeit: Mantisse enthält nur signifikante Stellen, keine leading/trailing zeros.
- Bei Fixed-Point mit zu hohem Wertebereich sind bei kleinen Zahlen die höherwertigen Bits verschwendet (alle 0).

Nutzen Charakteristik
- Gibt an, in welchem Wertebereich (Grössenordnung) die Zahl liegt.
- Verschiebt das binäre Komma: $128 = 1.0 \cdot 2^7$ und $0.25 = 1.0 \cdot 2^{-2}$ haben die gleiche Mantisse, nur die Charakteristik ist anders.

Gleitkomma vs. Festkomma
- \+ **Float:** grosser Wertebereich, flexibel; Multiplikation unproblematisch (Charakteristiken addieren).
- – **Float:** Rundungsfehler bei Addition stark unterschiedlicher Grössenordnungen, z.B. $2^{20} + 2^{-10} = 2^{20}$ (Bit fällt aus der 23-Bit-Mantisse). Gefahr: $f + \Delta f = f$, Δf klein.
- \+ **Fixed:** Format Resultats angepasst (I1Q4 · I1Q4 = I2Q8), mehr Bits für den Wert, keine FPU nötig.
- – **Fixed:** Wertebereich muss im Voraus gewählt werden.

17.2 IEEE 754: Addition / Subtraktion

Esempio: 0.74 + 0.09 = 0.83

Conversione: normalizzare nella forma $1.f$, convertire la parte frazionaria in binario moltiplicando ripetutamente per 2 (annotare il bit prima della virgola e continuare con il resto), arrotondare a 23 bit.

$0.74 = 1.48 \cdot 2^{-1} \quad \Rightarrow e = -1 + 127 = 126, s = 0$

$0.09 = 1.44 \cdot 2^{-4} \quad \Rightarrow e = -4 + 127 = 123, s = 0$

	Hex	Mantissa $1.f$
0.74	`3F3D70A4`	`1.0111 1010 1110 0001 0100 100`
0.09	`3DB851EC`	`1.0111 0000 1010 0011 1101 100`

1 Normalizzati? Entrambi i numeri hanno la forma $1.f$ ✓

2 Allineare gli esponenti. 0.09 ha l'esponente minore: $\Delta e = 126 - 123 = 3$. La mantissa viene spostata di 3 posizioni a **destra**, l'esponente aumenta di 3. I bit che escono (in grigio) si tengono come resto.

prima	2^{-4}	1.0111 0000 1010 0011 1101 100
dopo	2^{-1}	0.0010 1110 0001 0100 0111 101 100

3 Sommare le mantisse (portandosi dietro i bit di resto):

	2^{-1}	1.0111 1010 1110 0001 0100 100 000
+	2^{-1}	0.0010 1110 0001 0100 0111 101 100
=	2^{-1}	1.1010 1000 1111 0101 1100 001 100

4 Normalizzare. Somma < 2, quindi non serve nessuno spostamento.

5 Arrotondare (round to nearest)

2^{-1} 1.1010 1000 1111 0101 1100 010

6 Ritorno al formato IEEE. $e = -1 + 127 = 126$; l'1 iniziale di $1.f$ viene omesso.

s	e (8 bit)	f (23 bit)	Hex
0	0111 1110	101 0100 0111 1010 1110 0010	3F547AE2

Segni diversi: Le mantisse sono senza segno (il segno sta nel bit s). Quindi:
1. Confrontare i valori assoluti (**dopo** aver allineato gli esponenti).
2. Calcolare sempre **valore assoluto maggiore meno valore assoluto minore**
3. Segno del risultato = segno del numero con il valore assoluto maggiore.

$$|A| > |B| \Rightarrow \text{mantissa} = |A| - |B|, \quad s = s_A$$

17.3 IEEE 754: Moltiplicazione / Divisione

Esempio: $6.625 \cdot (-0.5) = -3.3125$

Conversione (stesso metodo dell'addizione):
$6.625 = 110.101_2 = 1.10101 \cdot 2^2 \quad \Rightarrow e = 2 + 127 = 129, s = 0$
$-0.5 = -1.0 \cdot 2^{-1} \quad \Rightarrow e = -1 + 127 = 126, s = 1$

1 Zero? Nessuno dei fattori è `0.0f` ✓

2 Sommare gli esponenti. Il bias 127 deve comparire **una sola volta**, quindi dopo la somma va sottratto una volta:

$e = e_1 + e_2 - 127 = 129 + 126 - 127 = 128 \quad \Rightarrow 2^{128-127} = 2^1$

3 Moltiplicare le mantisse, determinare il segno. Moltiplicare normalmente le mantisse $(1.f)$; segno: $s = s_1 \oplus s_2 = 0 \oplus 1 = 1$.

`1.10101 · 1.00000 = 1.10101`

4 Normalizzare. (Se ≥ 2: mantissa di 1 posizione a destra, esponente +1.)

5 Formato IEEE. Arrotondare, per il resto come nell'addizione.

s	e (8 bit)	f (23 bit)	Hex
1	1000 0000	101 0100 0000 0000 0000 0000	C0540000

18 Schnittstellen & Peripherie

18.1 μC-Peripherie

- Ankopplung über Systembus des μC's.
- Hardwaremässige Umsetzung.
- Control-, Daten- & Statusregister.
- Ein- bzw. Ausgabe über I/O-Ports.

18.2 Schieberegister

Parallel In Serial Out (PISO): Einfache Verkettung von Flip-Flops. Zusätzliche Multiplexer nötig.
Serial In Parallel Out (SIPO): Einfache Verkettung von Flip-Flops.

[Placeholder: Diagramme zu PISO / SIPO / Peripherie Blockschaltbild]

18.3 Taktgeber in einem μP-System

Wichtigkeit, den Taktgeber richtig zu wählen:
- Für Synchronität, Genauigkeit (Präzision), Strom/Effizienz.
- Clock soll so gewählt sein, dass alle zeitkritischen Anforderungen in der definierten Zeit abgearbeitet werden können.
- Performance vs. Leistungsverbrauch.

18.4 Direct Memory Access (DMA)

- Daten werden nicht mittels Load/Store über CPU transferiert, sondern direkt über den DMA.
- Daten werden nicht bearbeitet oder verändert, sie werden als ganze Pakete transferiert.
- Erst am Ende bzw. nach der Hälfte eines Datenpaket-Transfers wird ein Interrupt generiert. (non è obbligatorio generare interrupt, pheri ⇒pheri)
- Interrupt Stacking & Fetching wird reduziert.

Anwendungsbeispiele:
- *High Performance:* Datentransfer & Verarbeitung erfolgt in grossen Paketen. CPU kann Daten verarbeiten während der DMA transferiert.
- *Ultra Low Power:* Datentransfer & -Verarbeitung erfolgt in grossen Paketen. CPU ist im Standby Modus während der DMA transferiert.

18.5 UART

- Besitzt keinen Clk. (asynchrone Übertragung).
- Ausschliesslich Punkt-zu-Punkt Verbindung (Tx, Rx).
- Full-Duplex Betrieb uneingeschränkt möglich.
- Diverse Treiber (RS-232, RS-422, RS-485) für längere Distanzen.
- Stromsparend (Push-Pull-Ausgänge).

Synchronisation: Mittels des Start-Bits, welches bei jedem Symbol zu Beginn übertragen wird.
UART vs. RS-232: UART: Logik-Pegel 1 = High, 0 = Low.
RS-232: Schnittstellen-Pegel $1 = -3V \cdots -12V$, $0 = +3V \cdots +12V$.

18.6 SPI

Synchrone Schnittstelle mit minimalen Spezifikationen.
- Durch Synchronität höhere Übertragungsraten als mit UART möglich.
- Eingeschränkter Full-Duplex Betrieb oder Half-Duplex.
- Leitungen: CS, SCLK, MOSI, MISO.
- Mehrere Slaves am gleichen Interface. Nur kurze Distanzen möglich.
- Stromsparend (Push-Pull-Ausgänge).

SPI: SW Data-Ready bei Subs
- Problematik: Sub kann dem Main kein "Data Readyßenden, Main muss aktiv ein Status-Register abfragen.
- Vorteil: Keine zusätzlichen Leitungen.
- Nachteile: Ständige Abfrage des Status-Registers, erhöhter Datentransfer, kein effizienter Standby-Betrieb möglich.

SPI: HW Data-Ready bei Subs HW-Abfrage: Zusätzliche Interrupt-Leitungen und Data-Ready-Leitung.
- Vorteile: Keine überflüssiger Datentransfer, effizienter Standby-Betrieb möglich.
- Nachteil: Zusätzliche Leitungen zu SPI-Schnittstelle.

18.7 I2C

Synchrone Schnittstelle mit höheren Spezifikationen.
- Half-Duplex Bus Topologie.
- Nicht stromsparend (Open-Drain-Ausgänge mit Widerstand).

19 Anhang

Hex	A	B	C	D	E	F
Dec	10	11	12	13	14	15
Bin	1010	1011	1100	1101	1110	1111

Parallel: Baudrate: $f_{Baud} = \frac{1}{100\mu s} = 10'000$ Baud
Bitrate: $f_{Bit} = \frac{8}{100\mu s} = 80'000$ Bit/s
Seriell: Baudrate: $f_{Baud} = \frac{1}{100\mu s} = 10'000$ Baud
Bitrate: $f_{Bit} = \frac{1}{100\mu s} = 10'000$ Bit/s

Frequenzabweichung:
Damit die Funktion gewährleistet werden kann, darf bei der Übertragung eines Frames die maximale Abweichung höchstens 1 Bitbreite betragen.

$$f_{c,i} = \left(1 \pm \frac{\Delta t_{max}}{t_{max}}\right) * f_{2,i} = \left(1 \pm \frac{1\text{ Bit}}{2 \cdot \text{Framelänge}}\right) * f_{c,i}$$

[Placeholder: Diagramm / Text bezüglich ADC Rauschen und statistischer Verteilung (Messwerte 460.782)]